

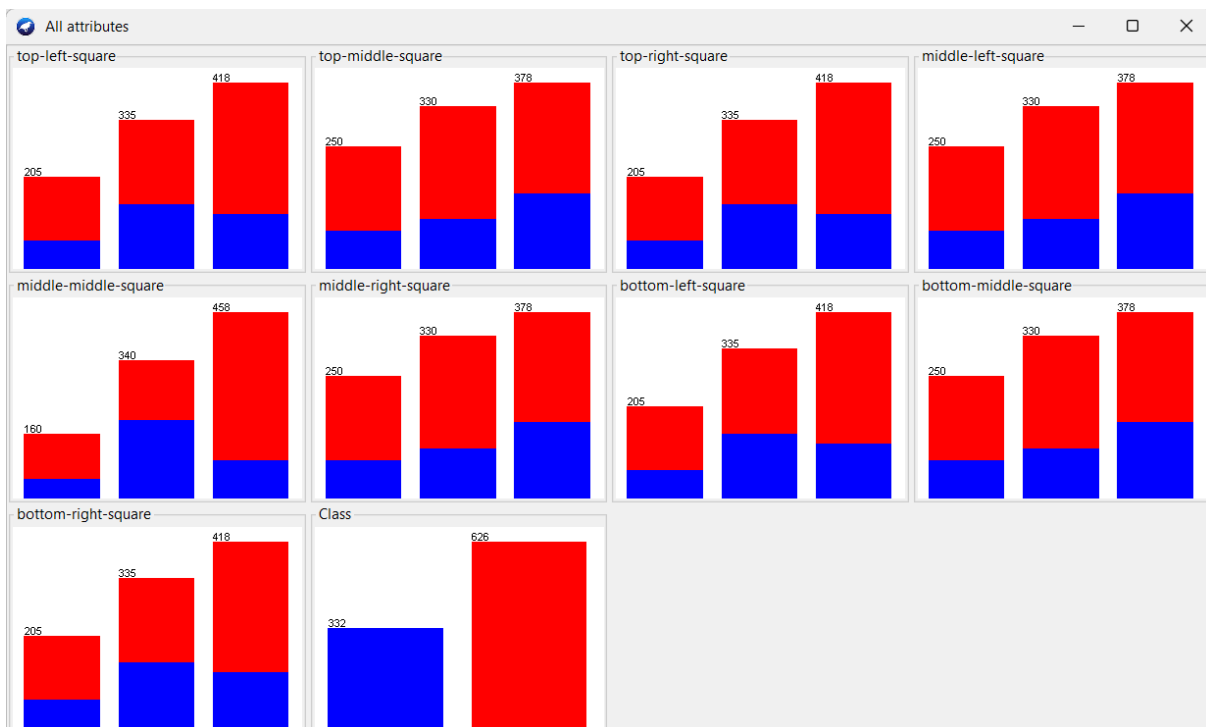
Assignment 4:

Machine Learning Laboratory

Name: Rishabh Ray
Student ID: 34525416

Question 1: Classification, Decision Trees, Naive Bayes, k-NN, WEKA

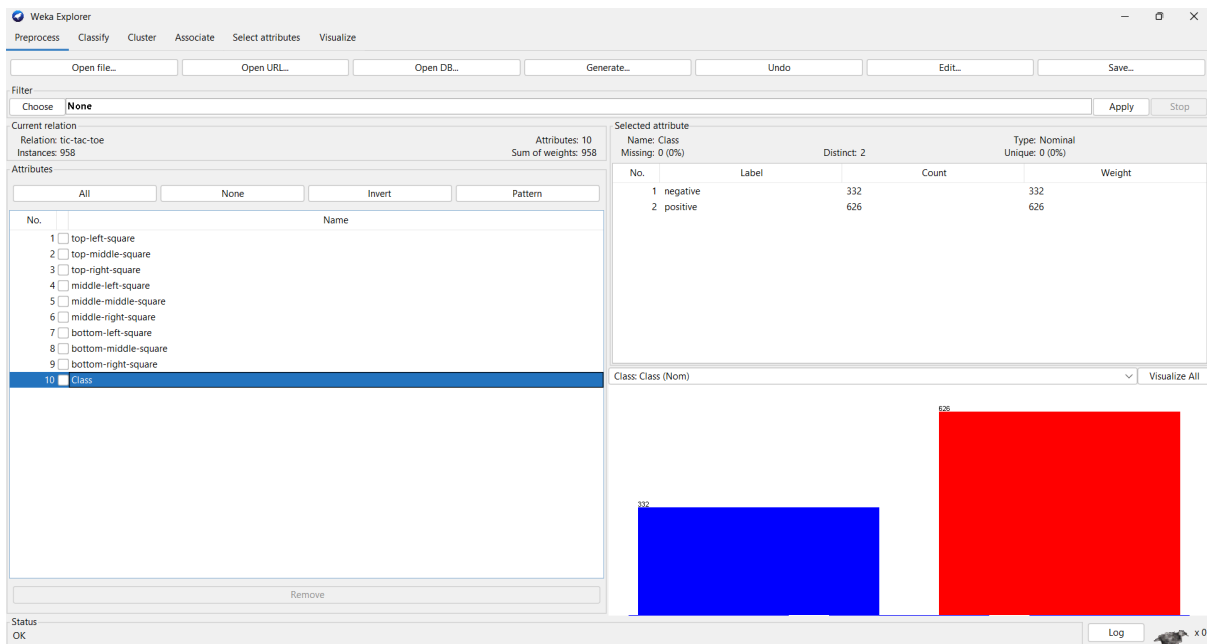
1.1. (a) Which attributes seem to be the most predictive of winning or losing?
(Hint: if you were the “x” player, where would you put your first cross and why?)



As we can see from this visualisation of all the cases, the most predictive attributes are:

1. **Middle-middle-square:** It is the most important square in the tic tac toe as it contributes to four winning lines, horizontal, vertical and both diagonals. We can see a similar inference from the visualisation where x has many positive instances when playing in the middle square.
2. **Top-right-square:** Virtually all corner squares are the 2nd most important in a tic-tac-toe game, but as per the visualisation, the bar for X in the top left square shows a significant win for blue when starting here
3. **Bottom-right square:** Similar to the top-right square, this corner square also has a high possibility of winning. The same inference can be drawn from the visualisation, where a significant portion of the graph is blue, indicating that X wins often when starting here.

1.1.(b) What can you infer about the advantage (or otherwise) of being the first player?



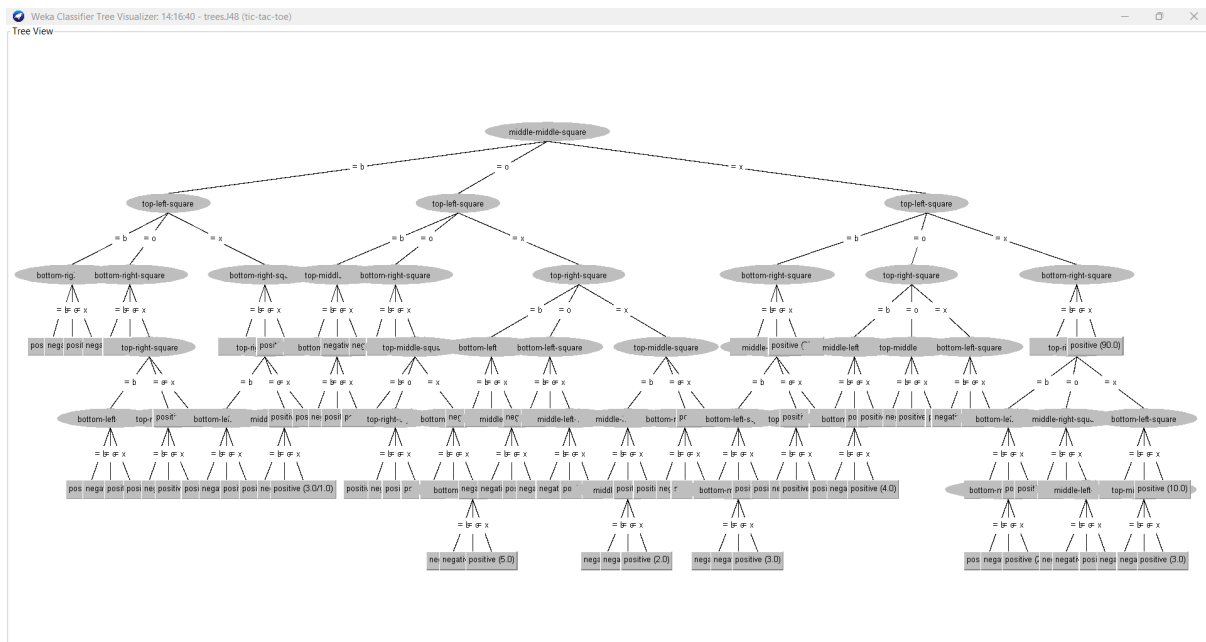
From the class distribution, we can see that the first player has 626 wins vs. 332 losses for the game. This is about a 65% win rate for being the first player. This is a big advantage for the player who is going first in the game, as he has more chances of either marking a high-winning probability square or blocking a winning square for the opponent.

1.2. Run J48 (=C4.5, Decision Tree), Naïve Bayes and IBk (=k-NN) to learn a model that predicts whether the “x” player will win. Perform 10-fold cross-validation, and analyse the results obtained by these algorithms as follows.

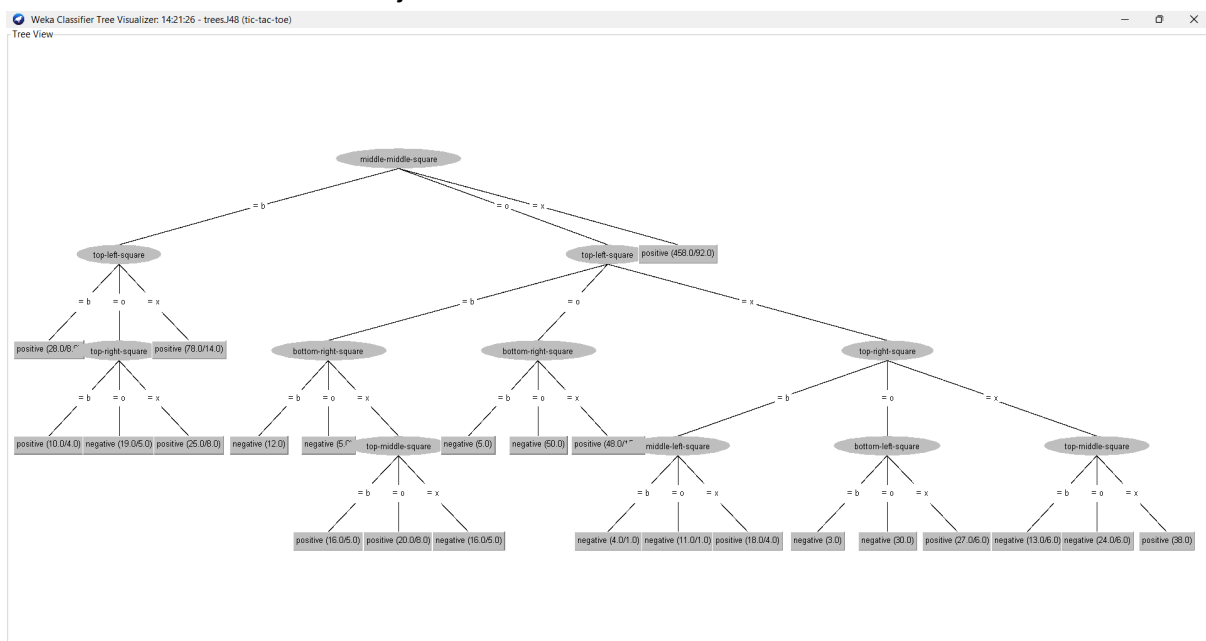
Note: When using J48, click on the “Choose” bar to try at least two values of minNumObj (default is 2); and when using IBk, try at least three values of KNN (default is 1).

1.2.(a) i. Examine the Decision Tree and indicate the main variables.

Decision Tree with minNumObj = 2 :



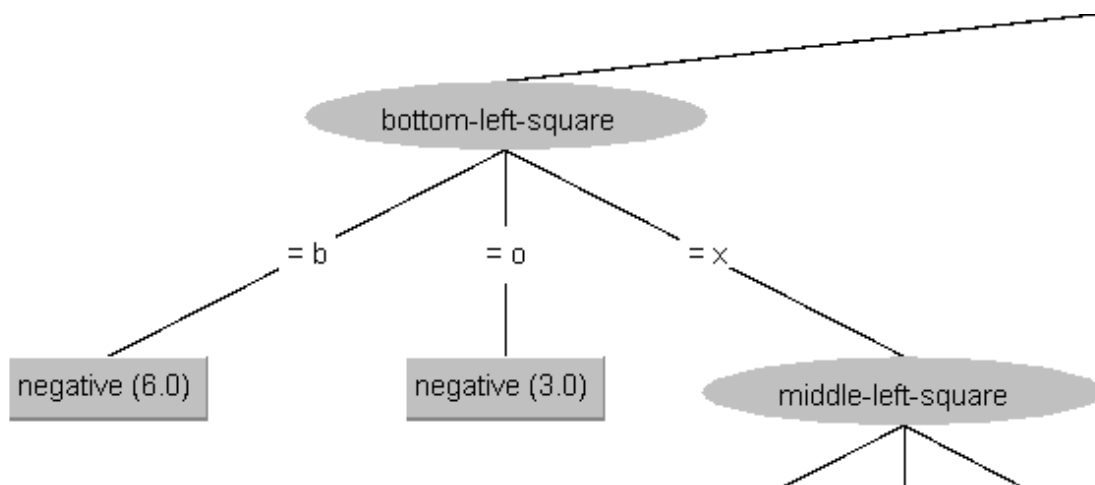
Decision Tree with minNumObj = 10



The **Main variables** that appear at the top of both decision trees are the same:

1. For the root variable, we have the **middle-middle-square**, which is due to the almost guaranteed win being played in the middle
2. Corner squares: **The top-left and bottom-right squares** are the second-most important squares after the middle for a win. They decide whether the first player can win with a diagonal or a fork.

1.2.(a) ii. Trace the Decision Tree for the following game. What would it predict? Does this prediction make sense?



As per the decision tree, when we trace this game, the output is negative, which suggests that the first player (x) is predicted to lose.

This is under our findings above, as in the majority of the cases, when the centre is held by the first player loses.

1.2.(a) iii. What is the first split in the Decision Tree? Calculate (by hand) the Information Gain obtained from the first split in the tree. Show your calculations.

The first split in both the decision trees was middle-middle-square as the root.

Now, to calculate the information, we need to first calculate:

Given total instances = 958

Positive instances (Wins) = 626

Probability of total wins = $626/958=0.653$

Negative instances (losses) = 332

Probability of total losses = $332/958=0.347$

Now we will calculate the entropy of the node before the split =

$$H(S) = -P(\text{positive}) \times \log_2(P_{\text{positive}}) - P(\text{negative}) \times \log_2(P_{\text{negative}})$$

$$H(S) = -(0.653) \times \log_2(0.653) - (0.347) \times \log_2(0.347)$$

$$H(S) = -0.653 \times (-0.616) - 0.347 \times (-1.572)$$

$$H(S) = 0.402 + 0.530 = 0.932$$

1.2.(a) iv. What is the accuracy of the Decision Tree? Explain the results in the confusion matrix for the best option you tried.

The screenshot shows the Weka Explorer interface with the Classifier tab selected. The classifier chosen is J48 - C 0.25 - M 10. The test options are set to Cross-validation with 10 folds. The classifier output is displayed, showing the tree structure and performance metrics.

Classifier output:

```

| | | | | top-middle-square = 0: negative (x.0)
| | | | | top-middle-square = x: positive (3.0)
| | | | | bottom-left-square = x: positive (10.0)
| | | | | bottom-right-square = x: positive (90.0)

```

Number of Leaves : 95
Size of the tree : 142
Time taken to build model: 0.02 seconds

==== Stratified cross-validation ====

==== Summary ====

Metric	Value
Correctly Classified Instances	810
Incorrectly Classified Instances	148
Kappa statistic	0.6574
Mean absolute error	0.1695
Root mean squared error	0.3439
Relative absolute error	37.4049 %
Root relative squared error	72.262 %
Total Number of Instances	958

==== Detailed Accuracy By Class ====

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
Weighted Avg.	0.768	0.113	0.782	0.768	0.775	0.658	0.896	0.835	negative
	0.887	0.232	0.878	0.887	0.882	0.658	0.896	0.923	positive

==== Confusion Matrix ====

	a	b	<-- classified as
255 77	a = negative		
71 555	b = positive		

For the decision tree with the best setting, which was the default minNumObj = 2, we found that:

Overall accuracy = 84.5511 %

The confusion matrix tells us:

	Predicted “lose”	Predicted “win”	Row total
Actual “lose”	255 (True Negatives)	77 (False Positives)	332
Actual “win”	71 (False Negatives)	555 (True Positives)	626

- **True positives (555)** – In 555/626 of winning boards, the tree correctly forecasts a win for the first (“x”) player.
- **True negatives (255)** – In 255/332 of losing boards, it correctly predicts a loss.
- **False positives (77)** – positions where the tree expects a win but the first player actually loses; these usually feature an x in the centre that is later blocked.
- **False negatives (71)** – rarer boards where the first player wins despite not holding the centre, a pattern the tree struggles to capture.

1.2. (b) i. Calculate (by hand), from the probability distributions in Weka's output, the predicted probability of a win and a loss for the game in item 2(a)ii. Show your calculations.

After running the model with NaiveBayes classification, we see Weka's output, now using that:

X	X	
○	○	X
	○	

From Weka, we see that: **P(negative)** = 0.35, **P(positive)** = 0.65.

Now, calculating the conditional probabilities for every square:

Square (value)	Probability of Loss	Probability of Win
top-left = x	124/335=0.370	296/629=0.471
top-middle = x	154/335=0.460	226/629=0.359
top-right = b	64/335=0.191	143/629=0.2277
middle-left = o	102/335=0.304	230/629=0.366
middle-middle = o	193/335=0.576	149/629=0.237
middle-right = x	154/335=0.460	226/629=0.359
bottom-left = b	64/335=0.191	143/629=0.227
bottom-middle = o	102/335=0.304	230/629=0.366
bottom-right = b	64/335=0.191	143/629=0.227

Now un-normalising the class products:

Loss (negative):

$$=0.35 \times (0.370)(0.460)(0.191)(0.304)(0.576)(0.460)(0.191)(0.304)(0.191) \\ =1.02 \times 10^{-5}$$

Win (positive):

$$=0.65 \times (0.471)(0.359)(0.227)(0.366)(0.237)(0.359)(0.227)(0.366)(0.227) \\ =1.47 \times 10^{-5}$$

Finding posterior probabilities:

$$P(\text{win}) = 1.47 \times 10^{-5} / (1.47 \times 10^{-5} + 1.02 \times 10^{-5}) = 0.590$$

$$P(\text{loss}) = 1.02 \times 10^{-5} / (1.47 \times 10^{-5} + 1.02 \times 10^{-5}) = 0.410$$

1.2. (b) ii. What is the accuracy of the Naïve Bayes classifier? Explain the results in the confusion matrix. What is the prediction of Weka's Naïve Bayes classifier for the game in item 2(b)i, and the probability of this prediction?

```
=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      667           69.6242 %
Incorrectly Classified Instances    291           30.3758 %
Kappa statistic                     0.2843
Mean absolute error                 0.3702
Root mean squared error            0.4319
Relative absolute error             81.7182 %
Root relative squared error        90.7504 %
Total Number of Instances          958

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
      0.428   0.161   0.584    0.428   0.494     0.291   0.744    0.635    negative
      0.839   0.572   0.734    0.839   0.783     0.291   0.744    0.857    positive
Weighted Avg.   0.696   0.430   0.682    0.696   0.683     0.291   0.744    0.780

=== Confusion Matrix ===
  a  b  <-- classified as
142 190 |  a = negative
101 525 |  b = positive
```

The accuracy of the Naive Bayes classifier is 69.6242, where 667 / 958 games are correctly classified.

The Confusion Matrix shows:

	Predicted lose	Predicted win	Row total
Actual lose	142 (TN)	190 (FP)	332
Actual win	101 (FN)	525 (TP)	626

This confusion matrix tells us that:

- The model recognises the wins well, where 525 of 626 games are recognised as actual wins.
- This model does not do well on predicting losses, where only 142 games out of 332 losses are expected as losses. The rest are mistaken for wins.

Prediction for the board in 2(a)ii

Using the conditional-probability table supplied by WEKA and the calculation done earlier, we see that the Naïve Bayes calculation gives

$$P(\text{win} | \text{board}) = 0.59, \quad P(\text{lose} | \text{board}) = 0.41$$

Hence, weka predicts “win” (positive class) with a probability = 59 % for that specific game position.

1.2 (c) i. Find three instances in the dataset that are similar to the game in item 2(a)ii, and use the Jaccard coefficient, combined with a distance metric, to calculate (by hand) the predicted outcome for this game. Show your calculations.

The three similar instances that I used are:

1. Plot: Master Plot

Instance: 102

top-left-square : x

top-middle-square : x

top-right-square : b

middle-left-square : x

middle-middle-square : o

middle-right-square : o

bottom-left-square : x

bottom-middle-square : o

bottom-right-square : b

Class: positive

2. Plot: Master Plot

Instance: 103

top-left-square : x

top-middle-square : x

top-right-square : b

middle-left-square : x

middle-middle-square : o

middle-right-square : o

bottom-left-square : x

bottom-middle-square : b

bottom-right-square : o

Class: positive

3. Plot: Master Plot

Instance: 109

top-left-square : x

top-middle-square : x

top-right-square : b

middle-left-square : o

middle-middle-square : x

middle-right-square : o

bottom-left-square : b

bottom-middle-square : o

bottom-right-square : x

Class: positive

Each of these instances differs from the query in just 2-4 squares, making them the required instance.

Now for Jaccard similarity and distance calculations:

For two games, A and B

$J(A,B) = (\text{non-blank matches})/(\text{positions where at least one is non-blank})$

$d(A,B) = 1 - J(A,B)$

	Non-blank matches (intersection)	Non-blank union	J	d
Query = 102	4 (sq 1, 2, 5, 8)	7	$4/7=0.5714/7=0.571$	0.429
Query = 103	3 (sq 1, 2, 5)	8	$3/8=0.3753/8=0.375$	0.625
Query = 109	4 (sq 1, 2, 3, 8)	7	$4/7=0.5714/7=0.571$	0.429

Now checking for the weights:

Weight(positive) = $0.571 + 0.375 + 0.571 = 1.517$

Weight(negative) = 0 (no negative neighbours)

As all the similarity weights are in the positive class, we can predict that:

X player wins the game.

1.2 (c). ii. What is the accuracy of the k-NN classifier? Explain the results in the confusion matrix.

Classifier output

```

bottom right square
Class
Test mode: 10-fold cross-validation

=== Classifier model (full training set) ===

IB1 instance-based classifier
using 5 nearest neighbour(s) for classification

Time taken to build model: 0 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      948           98.9562 %
Incorrectly Classified Instances     10            1.0438 %
Kappa statistic                    0.9768
Mean absolute error                 0.1909
Root mean squared error             0.2315
Relative absolute error             42.1333 %
Root relative squared error         48.6378 %
Total Number of Instances          958

=== Detailed Accuracy By Class ===

                TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                0.973    0.002    0.997     0.973    0.985     0.977    1.000    0.999    negative
                0.998    0.027    0.986     0.998    0.992     0.977    1.000    1.000    positive
Weighted Avg.   0.990    0.018    0.990     0.990    0.990     0.977    1.000    1.000

=== Confusion Matrix ===

  a  b  <-- classified as
323  9  |  a = negative
 1 625 |  b = positive

```

The accuracy of the KNN model is: 98.9562 %

Whereas the confusion matrix shows:

	Predicted "lose"	Predicted "win"	Row total
Actual "lose"	323 (True Negatives)	9 (False Positives)	332
Actual "win"	1 (False Negative)	625 (True Positives)	626

This confusion matrix shows us that:

- We have almost a near-perfect recall for wins (625/626)
- For the losses, we also see a very high recall score - (323/332)

Therefore, there is a very tiny error rate.

1.3. (5 + 3 = 8 marks) Draw a table to compare the performance of J48, Naive Bayes and IBk using the accuracy, recall, precision and F-score measures produced by Weka. Which algorithm does better? Explain in terms of these summary measures. Can you speculate why?

Algorithm	Overall Accuracy	Positive ("win")			Negative ("lose")			Weighted Avg.
		<i>Precision</i>	<i>Recall</i>	<i>F-score</i>	<i>Precision</i>	<i>Recall</i>	<i>F-score</i>	<i>F-score</i>
J48 (minNumObj = 2)	84.6 %	0.878	0.887	0.882	0.783	0.768	0.775	0.844
Naïve Bayes	69.6 %	0.734	0.839	0.783	0.584	0.428	0.494	0.683
KNN (k = 1–5)	99.0 %	0.986	0.998	0.992	0.997	0.973	0.985	0.990

In this table, we observe that:

- **KNN** is the most accurate algorithm as it has around 99% overall accuracy. This is likely because tic-tac-toe final boards are "lookup-table friendly" and storing the exact cases of the games is easier for KNN algorithm.

Question 2: Classification, Decision Trees, Naive Bayes, k-NN, WEKA (36 marks)

Consider the dataset `postoperative-patient-data_simplified.arff` available on Moodle.

This dataset contains health-status attributes of post-operative patients in a hospital, with the target class being whether the patients should be discharged (S) or remain in the hospital (A). Additional documentation regarding these attributes appears in the ARFF file.

2.1. Before you run the classifiers, use the Weka visualisation tool to analyse the data, and report briefly on the types of different variables and on the variables that appear to be important. (4 marks)

Attribute (predictor)	Importance
L-CORE (core temp)	Checking the core temperature is important as extremes of core temperature favour keeping the patient.
L-SURF (surface temp)	The surface temperature helps to reinforce the core temperature finding.
L-O2 (oxygen saturation)	Checking the oxygen level helps in prediction as lower O ₂ is a risk factor → likely to “stay”.
L-BP (blood pressure level)	BP is also important as abnormal BP levels predict “stay”.
SURF-STBL (surface-temp stability)	Stability is temperature helps determine the stability of the patient
CORE-STBL (core-temp stability)	Core temperature is also a strong predictor, as too much temperature variation is not good for the patient.
BP-STBL (blood-pressure stability)	Blood pressure fluctuation also hurts the patient’s health.
decision (class)	Target to predict.

2.2. Run J48 (=C4.5, Decision Tree), Naive Bayes and IBk (k-NN) to learn a model that predicts whether a patient should be discharged. Perform 10-fold cross-validation, and analyse the results

obtained by these algorithms as follows.

Note: Click on the “Choose” bar to select different values of the following parameters for J48

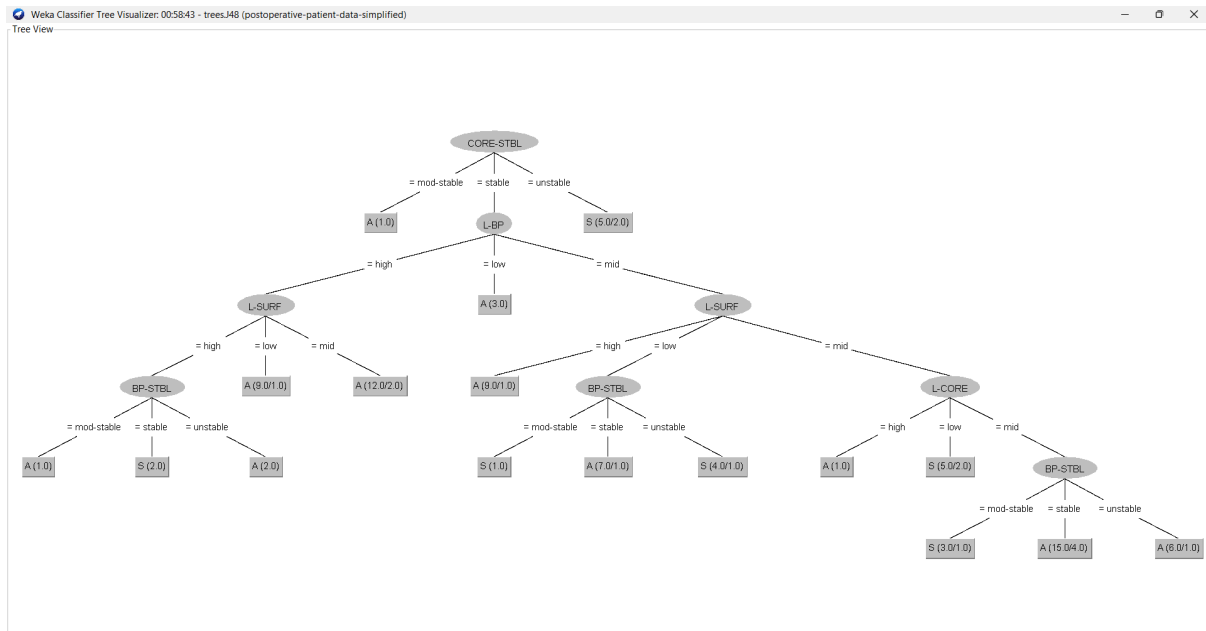
and k-NN (parameter variations of Naive Bayes are not relevant to this lab):

- **J48 – minNumObj** (at least two values).
- **k-NN (IBk, under lazy in weka), KNN** (at least two values) and **distanceWeighting** (at least two options).

Explain the meanings of these parameters. You should report on performance for at least two variations in the total of the operational parameter minNumObj for J48, and at least two variations of each KNN and distanceWeighting for k-NN (four variations in total for k-NN).

- **minNumObj:** This controls pruning: a split is kept only if every resulting leaf still has at least minNumObj training cases.
- **KNN:** This shows how many nearest training cases vote for a prediction. $K = 1$ memorises exact matches; larger K smooths over noise but may blur sharp class boundaries.
- **DistanceWeighting:** Tells WEKA how to weight each of the K votes.
 - None = unweighted majority vote.
 - Inverse distance = closer neighbours count more.
 - Similarity = uses $1-d$ instead of $1/d$, best when distances are already between 0 and 1 (e.g., Jaccard).

2.2.(a) i. Examine Weka's output (e.g., Decision Tree), and indicate which are the main variables.



From this decision tree, we can see that:

The most important attribute is:

1. **CORE-STBL (core-temperature stability)** - As it is the root node, it carries the most information gain.
2. **L-BP (blood-pressure level)** - This appears directly under the “stable” branch of CORE-STBL and decides whether the patient can be discharged based on BP level.

2.2.(a) ii. What is the accuracy of the output produced by Weka (e.g., Decision Tree)? Why is it different from the accuracy you would expect by considering only the majority class? Explain the results in the confusion matrix.

Classifier output

```

| | | | BP-STBL = mod stable: A (15.0/4.0)
| | | | BP-STBL = unstable: A (6.0/1.0)
CORE-STBL = unstable: S (5.0/2.0)

Number of Leaves : 17
Size of the tree : 25

Time taken to build model: 0 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances 60 69.7674 %
Incorrectly Classified Instances 26 30.2326 %
Kappa statistic 0.1387
Mean absolute error 0.3774
Root mean squared error 0.4957
Relative absolute error 93.1217 %
Root relative squared error 110.3555 %
Total Number of Instances 86

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall  F-Measure  MCC  ROC Area  PRC Area  Class
      -----
      0.871  0.750  0.750  0.871  0.806  0.147  0.569  0.726  A
      0.250  0.125  0.429  0.250  0.316  0.147  0.569  0.374  S
Weighted Avg. 0.698 0.577 0.660 0.698 0.669 0.147 0.569 0.628

=== Confusion Matrix ===
      a  b  <-- classified as
54  8 | a = A
18  6 | b = S

```

From this result, we can see that:

The overall accuracy is: 69.7674 %.

This is different from the accuracy expected by considering only the majority of the class.s

- Actual A (discharge) = 62 cases (72 %)
 - Actual S (stay) = 24 cases (28 %)
- B. Because the dataset is imbalanced, the naïve Bayes rule that always predicts A would therefore score 72 % accuracy.

The decision tree's 69.8 % is slightly lower because it deliberately trades some "easy" discharge predictions for the chance to recognise stay cases.

From the confusion matrix, we can see that:

	Predicted A	Predicted S	Row total
Actual A	54 (true discharges)	8 (kept in error)	62
Actual S	18 (missed stays)	6 (true stays)	24

We observe that:

- High recall for A: $54 / 62 \approx 87\%$ – the tree still discharges most patients who should leave.
- Low recall for S: $6 / 24, \approx 25\%$ – only one quarter of patients who need to stay are correctly identified.

2.2.(b) Naïve Bayes (2 + 8 + 3 = 13 marks)

2.2.(b) i. Explain the meaning of the “probability distributions” in Weka’s output, illustrating it concerning the BP-STBL attribute.

Note: Weka does smoothing when computing probabilities for Naive Bayes.

For each attribute, WEKA lists the Laplace-smoothed counts that it converts into the conditional probabilities $P(\text{attribute}|\text{Class})$ needed by Naive Bayes.

For instance:

BP-STBL		
mod-stable	18.0	5.0
stable	31.0	16.0
unstable	16.0	6.0
[total]	65.0	27.0

Here, the class totals are 65 and 27, not the raw 62 and 24, because WEKA adds one pseudo-case to every (value, class) pair (“add-1” or Laplace smoothing) to avoid zero probabilities.

2.2.(b) ii. Calculate (by hand), from the probability distributions in Weka's output, the probability that a person with the following attribute values would be discharged, and the probability that they would remain in hospital. Show your calculations.

L-CORE = mid

L-SURF = low

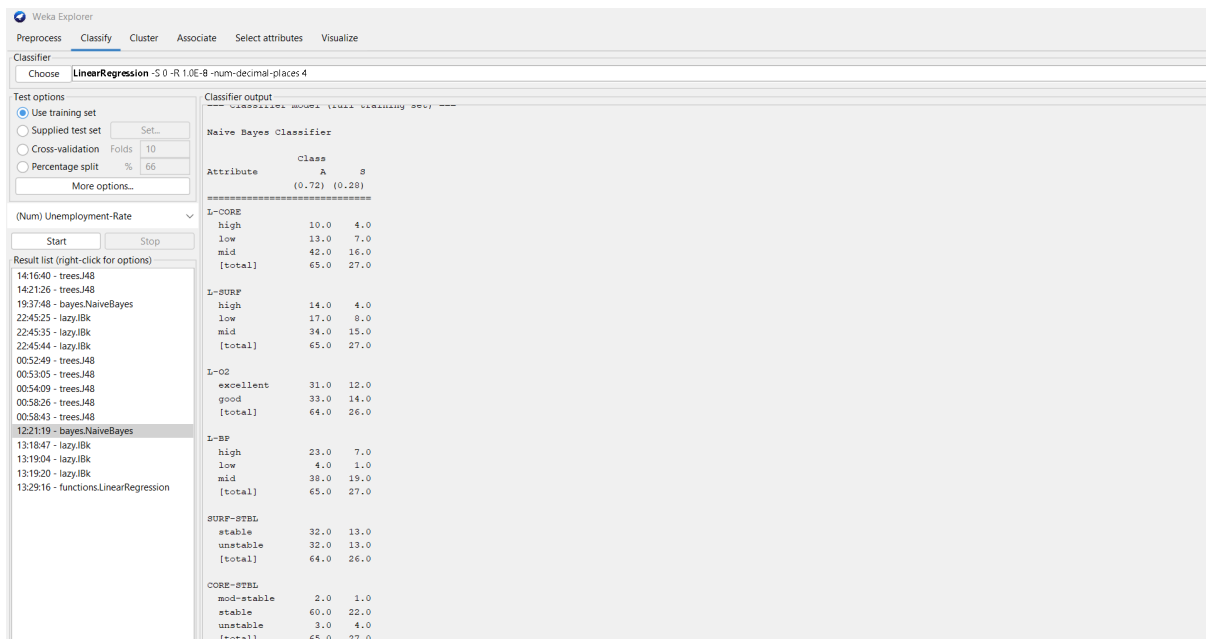
L-O2 = good

L-BP = high

SURF-STBL = stable

CORE-STBL = stable

BP-STBL = mod-stable



Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Classifier

Choose LinearRegression -S 0 -R 1.0E-8 -num-decimal-places 4

Test options

☒ Use training set ☐ Supplied test set ☐ Cross-validation ☐ Percentage split

Folds 10 % 66

More options...

(Num) Unemployment-Rate

Start Stop

Result list (right-click for options)

141640 - treesJ48
142126 - treesJ48
193748 - bayes.NaiveBayes
224525 - lazyJk
224535 - lazyJk
224544 - lazyJk
005249 - treesJ48
005305 - treesJ48
005409 - treesJ48
005826 - treesJ48
005843 - treesJ48
122119 - bayes.NaiveBayes
131847 - lazyJk
131904 - lazyJk
131920 - lazyJk
132916 - functions.LinearRegression

Classifier output

Naive Bayes Classifier

Class

Attribute A S

(0.72) (0.28)

L-CORE

high 10.0 4.0
low 13.0 7.0
mid 42.0 16.0
[total] 65.0 27.0

L-SURF

high 14.0 4.0
low 17.0 8.0
mid 34.0 15.0
[total] 65.0 27.0

L-O2

excellent 31.0 12.0
good 33.0 14.0
[total] 64.0 26.0

L-BP

high 23.0 7.0
low 4.0 1.0
mid 38.0 19.0
[total] 65.0 27.0

SURF-STBL

stable 32.0 13.0
unstable 32.0 13.0
[total] 64.0 26.0

CORE-STBL

mod-stable 2.0 1.0
stable 60.0 22.0
unstable 3.0 4.0
[total] 65.0 27.0

Class prior probability (from WEKA, after Laplace smoothing):

$P(A) = 0.72$, $P(S) = 0.28$

Conditional probabilities of the given attributes:

Attribute (value)	P of remaining in hospital	P of discharge
L-CORE = mid	$42/65=0.646$	$16/27=0.593$
L-SURF = low	$17/65=0.262$	$8/27=0.296$
L-O2 = good	$33/64=0.516$	$14/26=0.538$
L-BP = high	$23/65=0.354$	$7/27=0.259$
SURF-STBL = stable	$32/64=0.500$	$13/26=0.500$
CORE-STBL = stable	$60/65=0.923$	$22/27=0.815$
BP-STBL = mod-stable	$18/65=0.277$	$5/27=0.185$

To obtain the un-normalised joint probability for a given record, multiply the class prior $P(c)$ by the product of all attribute-wise conditional probabilities $P(x|c)$:

$$P(x,A) = 0.72 \times 0.646 \times 0.262 \times 0.516 \times 0.354 \times 0.500 \times 0.923 \times 0.277 \approx 0.00284$$

$$P(x,S) = 0.28 \times 0.593 \times 0.296 \times 0.538 \times 0.259 \times 0.500 \times 0.815 \times 0.185 \approx 0.00052$$

The posterior probability of class c for a record x is obtained by dividing its un-normalised joint probability by the sum of the joint probabilities for all classes.

$$P(A|x) = 0.00284 / (0.00284 + 0.00052) = 0.85$$

$$P(S|x) = 0.00052 / (0.00284 + 0.00052) = 0.15$$

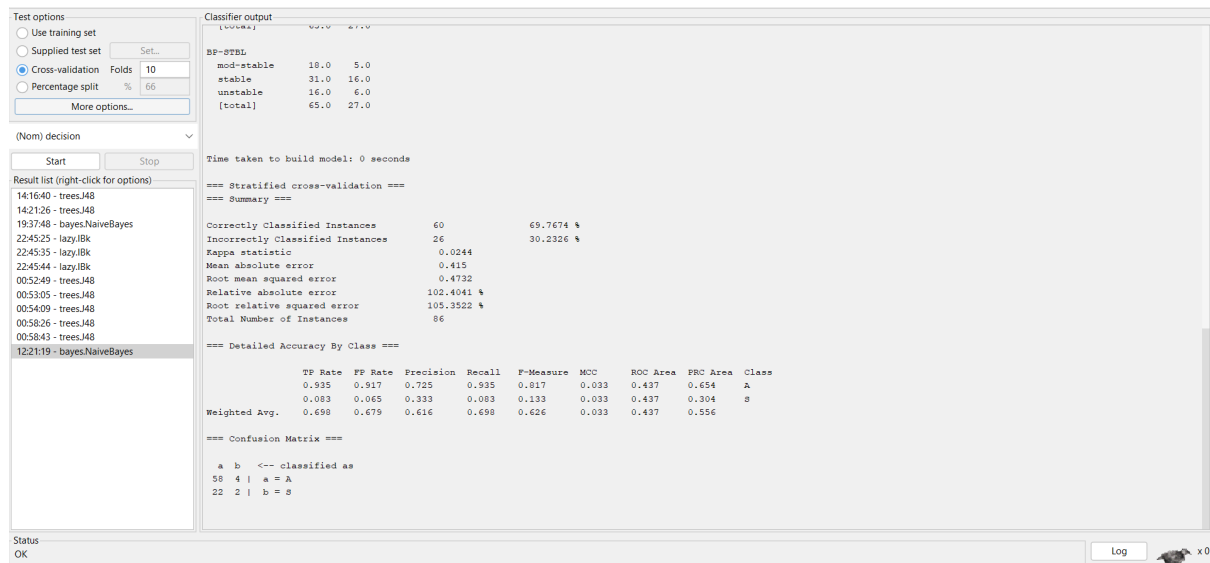
Now we get the result that:

Probability of discharge (A): = 85 %

Probability of remaining (S): = 15 %

Thus, Naïve Bayes would predict that this patient should be **discharged**.

2.2.(b) iii. What is the accuracy of the Naïve Bayes classifier? Explain the results in the confusion matrix. What is the prediction of Weka's Naïve Bayes classifier for the patient in item 2(b)ii, and the probability of this prediction?



The overall accuracy of this Naive Bayesian algorithm is 69.7674 %, which is mainly because Naïve Bayes improves only a little over random. After all, it models each attribute independently; that assumption is weak for strongly correlated physiological variables, so many stay cases are mis-scored.

The confusion matrix shows that:

	Predicted A(Discharge)		Predicted S(Stay)		Row total
Actual A Discharge)	58	True-Discharge (TD)	4	False-Stay (FS)	62
Actual S (Stay)	22	False-Discharge (FD)	2	True-Stay (TS)	24

This shows:

- **True-Discharge (58)** – patients who are correctly cleared to leave.
- **False-Stay (4)** – patients who are healthy enough to leave but are needlessly kept in hospital
- **False-Discharge (22)** – patients who needed further care but were predicted safe to leave; these are the most serious errors.
- **True-Stay (2)** – patients correctly flagged to remain for monitoring/treatment.

Prediction for the patient in 2(b) ii :

$$P(A|x) = 0.00284/(0.00284+0.00052) = 0.85$$

$$P(S|x) = 0.00052/(0.00284+0.00052) = 0.15$$

Now we get the result that:

Probability of discharge (A): = 85 %

Probability of remaining (S): = 15 %

Thus, Naïve Bayes would predict that this patient should be discharged.

2.2.(c) k-NN (6 + 2 = 8 marks)

2.2. (c) i. find three instances in the dataset that are similar to the patient in item 2(b), ii (you can do this visually), and use the Jaccard coefficient combined with a distance metric, to calculate (by hand) the predicted outcome for this patient. Show your calculations.

The instances I chose were:

Plot : Master Plot Instance: 12 L-CORE : mid L-SURF : low L-O2 : good L-BP : high SURF-STBL : stable CORE-STBL : stable BP-STBL : mod-stable decision : A

Plot : Master Plot Instance: 57 L-CORE : low L-SURF : low L-O2 : good L-BP : high SURF-STBL : stable CORE-STBL : stable BP-STBL : stable decision : A

Plot : Master Plot Instance: 7 L-CORE : mid L-SURF : low L-O2 : excellent L-BP : high SURF-STBL : stable CORE-STBL : stable BP-STBL : mod-stable decision : S

Now calculating the Jaccard's coefficient and distance metric:

$$J(A,B) = |A \cap B| / |A \cup B|$$

Pair	Matches	J	d=1-J
Instance = 12	7	$7/7=1.000$	0.000
Instance = 56	6	$6/7 \approx 0.857$	0.143
Instance = 7	6	$6/7 \approx 0.857$	0.143

Now, similarity-weighted 3-NN vote

$$\text{Weight(A)} = 1.000 + 0.857 = 1.857$$

$$\text{Weight(S)} = 0.857$$

Because the total similarity weight for A (discharge) exceeds that for S (stay):

K-NN (k = 3, Jaccard weighting) predicts the patient will be **discharged**.

2.2. (c) ii. What is the accuracy of the k-NN classifier for different values of k (kNN)? Explain the results in the confusion matrix.

k	Accuracy	Confusion matrix (rows = actual, columns = predicted)	Key points
1	61.6% (53 / 86)	A → A 53 A → S 9 S → A 24 S → S 0	With only one neighbour, the classifier copies the class of the single closest case. That correctly clears 53/62 discharge patients, but it also mistakes all 24 “stay” patients for “discharge” and even mislabels 9 safe patients as “stay.”
3	69.8% (60 / 86)	A → A 60 A → S 2 S → A 24 S → S 0	Majority vote over three neighbours suppresses the odd “stay” prediction for safe patients (only two left) and lifts accuracy by 8 %. The model still never recognises the stay class.
5	70.9% (61 / 86)	A → A 61 A → S 1 S → A 24 S → S 0	A larger neighbourhood removes the last false “stay,” giving the best accuracy of the three settings (but only one point better than $k = 3$). Recall for the stay class remains 0 %.

2.3. (3 + 3 = 6 marks) Draw a table to compare the performance of J48, Naive Bayes and IBk using the accuracy, recall, precision and F-score measures produced by Weka. Which algorithm does better? Explain in terms of these summary measures. Can you speculate why?

Model (10-fold CV)	Overall Accuracy	Precision / Recall / F-score(class A = discharge)	Precision / Recall / F-score(class S = stay)	Weighted F-score
J48(minNumObj 2)	69.8 %	0.75 / 0.87 / 0.81	0.43 / 0.25 / 0.32	0.67
Naïve Bayes	69.8 %	0.73 / 0.94 / 0.82	0.33 / 0.08 / 0.13	0.63
k-NN $k = 5$	70.9 %	0.72 / 0.98 / 0.83	0.00 / 0.00 / 0.00	0.60

- Best Accuracy is shown by the k--NN algorithm; however, it never recognises a single stay patient (recall $S = 0$), so its success is entirely on the majority class.
- The best balanced performance is shown by the decision tree algorithm(J48) as it sacrifices two points of accuracy but retains non-zero precision and recall for the minority 'stay' class, giving the highest weighted F-score (0.67).
- Naïve Bayes sits between the two as it has the same accuracy as J48, but is weaker on the stay class, so a lower weighted F-score.

Question 3: Regression (2 + 1 + 5 + 1 + 3 = 12 marks)

Consider the dataset **abs. ARFF** available on Moodle. This dataset contains continuous-valued economic attributes of a country, with the target variable being the unemployment rate. Additional documentation regarding these attributes appears in the ARFF file.

3.1. Perform a linear regression (Choose → functions → LinearRegression in Weka) to learn a linear model of the unemployment rate as a function of the other variables. You can use the default parameters given in Weka. What is the resultant regression function?

```
=== Classifier model (full training set) ===

Linear Regression Model

Unemployment-Rate =

    -0.0014 * All-Ords-Index +
    -0.2452 * Housing-Loan-Interest-Rate +
    13.7286

Time taken to build model: 0.06 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correlation coefficient          0.7002
Mean absolute error             1.1688
Root mean squared error         1.3724
Relative absolute error         73.2256 %
Root relative squared error     71.3913 %
Total Number of Instances      26
Ignored Class Unknown Instances          4
```

On running a linear regression algorithm on the **abs.arf** , I found the resulting regression to be this:

$$\text{Unemployment-Rate} = -0.0014 \times \text{All-Ords-Index} - 0.2452 \times \text{Loan-Interest-Rate} + 13.7286$$

Where:

Intercept: 13.7286

Slope for All-Ords-Index: -0.0014

Slope for Housing-Loan-Interest-Rate: -0.2452

3.2. Using the resultant regression function, calculate by hand the Absolute Error for the year 1986

For the Absolute Error in 1986, we can see the data from the Excel, which is:

	A	B	C	D	E	F	G
1	Year	Immigrants- ks	All-Ords- Index	Housing-Loan- Interest-Rate	Exports-ms	Unemployment- Rate	
2	1981	210.6	?	?	54375	6.1	
3	1982	172.7	?	?	54658	9.3	
4	1983	145.3	?	?	58829	10	
5	1984	163.2	855.1	?	12	8.8	
6	1985	186.4	1210.8	?	70481	8	
7	1986	204.4	1779.1	15.5	77547	8.4	
8	1987	242.3	1585.3	13.5	84101	7.7	
9	1988	249.9	1527.7	17	84981	6.5	

Now, using the regression model

$$y = 13.7286 - 0.0014 (\text{All-Ords Index}) - 0.2452 (\text{Housing-Loan Interest Rate})$$

And now for predicted values:

$$y(1986) = 13.7286 - 0.0014 \times (1779.1) - 0.2452 (15.5)$$

$$y = 13.7286 - 2.4907 - 3.8006$$

$$y = 7.4373$$

For calculating absolute error:

$$| y(1986) - y | = 7.4373 - 8.4$$

$$= 0.963\%$$

3.3. Calculate (by hand) the Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) obtained by the regression function between the years 1986 and 2006 (to avoid “?” values). You can use the Excel spreadsheet abs.xlsx provided on Moodle. How is MAE different from RMSE? (Do these functions emphasise different aspects of performance?)

Year	All-Ords Index	Loan rate (%)	Actual y	Predicted y	Error	Error ²
1986	1 779.1	15.5	8.40	7.437	0.963	0.928
1987	1 585.3	13.5	7.70	8.199	0.499	0.249
1988	1 527.7	17.0	6.50	7.421	0.921	0.849
1989	1 508.8	16.5	6.00	7.571	1.571	2.466
1990	1 504.9	13.0	8.50	8.434	0.066	0.004
1991	1 652.7	10.5	11.10	9.494	1.606	2.582
1992	1 722.6	9.5	11.80	9.768	2.032	4.129
1993	2 040.2	8.75	11.30	10.425	0.875	0.765
1994	2 000.8	10.5	9.30	9.643	0.343	0.118
1995	2 231.7	9.75	8.80	9.382	0.582	0.339
1996	2 662.7	7.20	8.90	9.249	0.349	0.122
1997	2 608.2	6.70	8.60	9.237	0.637	0.406
1998	2 963.0	6.50	7.80	9.200	1.400	1.960
1999	3 115.9	7.80	6.80	9.119	2.320	5.385
2000	3 352.4	6.80	6.60	9.135	2.535	6.426
2001	3 241.5	6.55	7.00	9.139	2.139	4.577
2002	3 032.0	6.55	6.40	9.042	2.642	6.978
2003	3 499.8	7.05	5.80	9.145	3.345	11.191
2004	4 197.5	7.30	5.20	9.619	4.419	19.531
2005	4 933.5	7.55	4.90	10.568	5.668	32.138
2006	6 337.6	8.05	4.50	12.022	7.522	56.569

The regression function, as we know from part 1, is:

- $y \text{ (unemployment-rate)} = 13.7286 - 0.0014 \text{ (All-Ords Index)} - 0.2452 \text{ (Housing-Loan Interest Rate)}$
- $\text{Error} = |\text{Predicted unemployment rate} - \text{actual unemployment rate}|$

Now the Mean Absolute Error for this is:

MAE = sum of all errors/number of errors

Sum of all errors = 21.55

Number of errors = 21

$\text{MAE} = 21.55/21 = 1.026$

For Root Mean Squared Error

RMSE = root of the (sum of all squared errors/number of errors)

sum of all squared errors = 34.53

Number of errors = 21

$\text{RMSE} = \text{root of } (34.53/21) = 1.28$

MAE and RMSE are different:

- MAE averages the magnitude of the errors linearly; every percentage-point miss contributes equally.
- RMSE squares each error before averaging, so large errors are penalised more heavily.

3.4. Use your model to predict the unemployment rate for the year 2010

Predicted unemployment rate for 2010:

Now using the regression model from part 1:

$$y = 13.7286 - 0.0014 (\text{All-Ords Index}) - 0.2452 (\text{Housing-Loan Interest Rate})$$

30	2009	605.1	4467.5	7.4	253762	?
31	2010	590.4	4607.1	7.8	254367	?
32						

For 2010:

All-Ords Index = 4 607.1

Housing-Loan Interest Rate = 7.8 %

Now Calculation:

$$y = 13.7286 - 0.0014 (4607.1) - 0.2452 (7.8)$$

$$y = 13.7286 - 6.4499 - 1.9126$$

$$y = 5.37\%$$

Predicted Unemployment Rate for 2010: 5.37%